

# Lidar Obstacle Avoidance

For instructions on app connection, please consult the tutorial located in “1.Getting Ready/1.4 APP Control”.

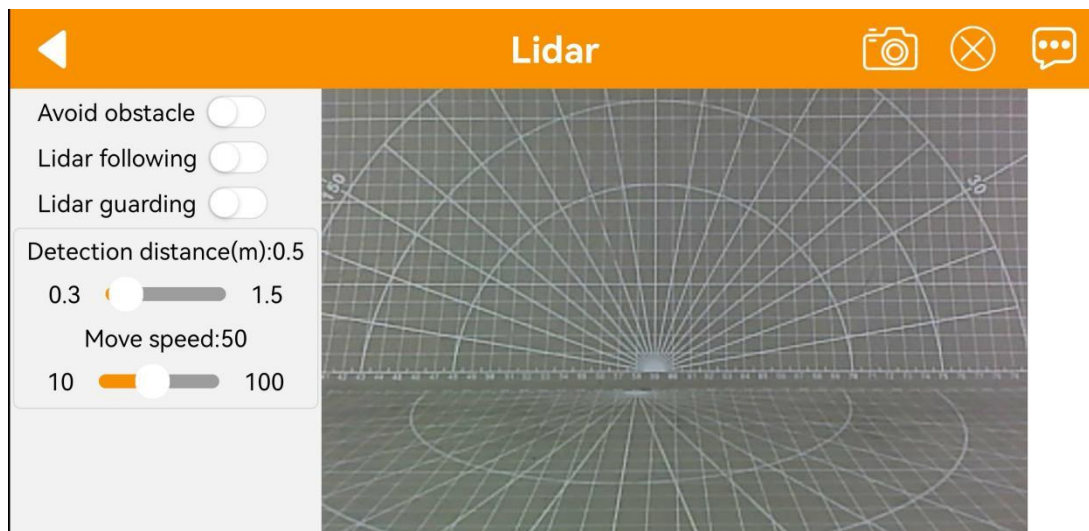
## 1. Enable Game

### 1.1 Initiate Lidar Game through App

- 1) Connect the robot to “WondePi” app.
- 2) Click-on Lidar to navigate to the game interface.



- 3) Switch on “Avoid obstacle” button to start the game.



### 1.2 Initiate Lidar Game Using Command

- 1) Start the robot, and access the robot system desktop using VNC according

to the tutorial saved in “8.Set Development Environment/Lesson 1 VNC Installation and Connection”.



2) Click-on  to open the command line terminal.

3) Execute the command and press Enter to deactivate the automatic start service for the app.

```
~/stop_ros.sh
```

```
> ~/stop_ros.sh  
zsh: killed ~/stop_ros.sh
```

4) Run the command to enable the local services for app-related game and chassis control services.

```
ros2 launch app lidar_node.launch.py debug:=true
```

```
> ~/stop_ros.sh  
zsh: killed ~/stop_ros.sh  
[KILL x | 09:53:18] ros2 launch app lidar_node.launch.py debug:=true
```

5) Open a new terminal, and execute the command. Then, press Enter to start the Lidar game.

```
ros2 service call /lidar_app/enter std_srvs/srv/Trigger {}
```

```
> ros2 service call /lidar_app/enter std_srvs/srv/Trigger {}  
waiting for service to become available...  
requester: making request: std_srvs.srv.Trigger_Request()  
response:  
std_srvs.srv.Trigger_Response(success=True, message='enter')
```

Note: The robot's performance initiated by the app and command is identical.

6) Enter the command and hit Enter to launch Lidar obstacle avoidance game.

```
ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 1}"
```

```
> ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 1}"  
waiting for service to become available...  
requester: making request: interfaces.srv.SetInt64_Request(data=1)  
response:  
interfaces.srv.SetInt64_Response(success=True, message='set_running')
```

7) If you want to exit the game, enter the command and press “Enter”.

```
ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 0}"
```

```
> ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 0}"
requester: making request: interfaces.srv.SetInt64_Request(data=0)

response:
interfaces.srv.SetInt64_Response(success=True, message='set_running')
```

Note: The game will continue to run under the current Raspberry Pi power-on state if not exited. To avoid excessive use of the Raspberry Pi's operating memory, please follow the above instructions to close the current game before executing other games.

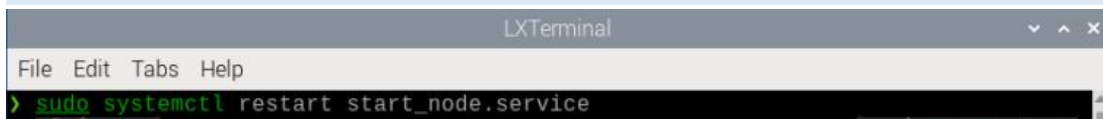
8) If you need to terminate the program, press short-cut "Ctrl+C" on the terminal opened in steps 4) and 5).

After experiencing the Lidar game, you can activate the app service either by using a command or restarting the robot. If the app service is not activated, related app functions will be disabled. In the case of a robot restart, the app service will start automatically.

Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

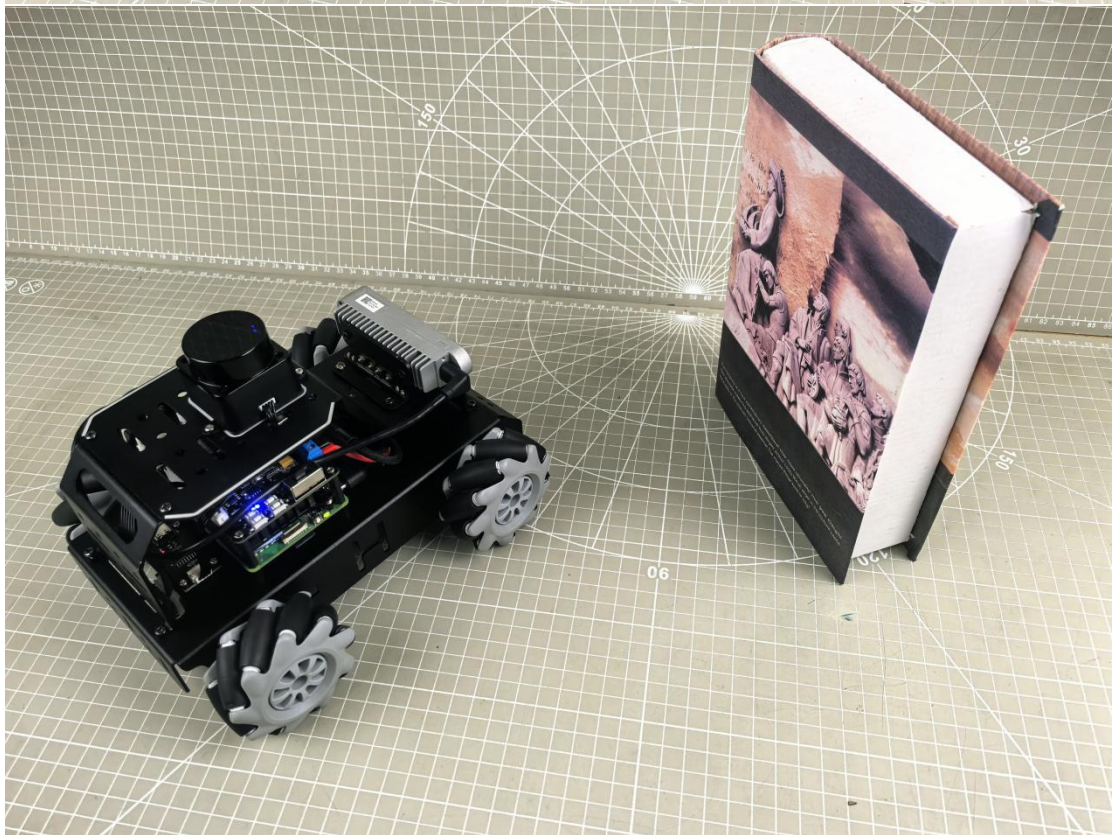
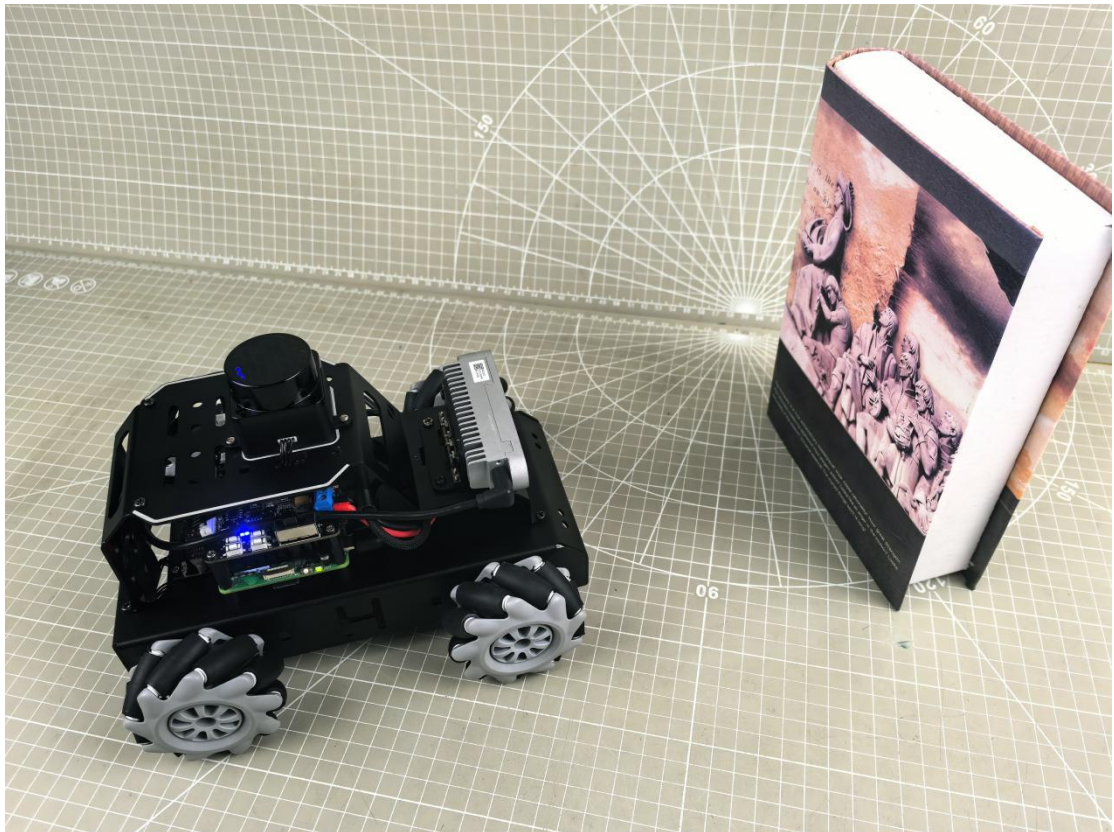
Note: please enter the command in the system path, not in the Docker container.

```
sudo systemctl restart start_node.service
```



## 2. Program Outcome

Let's use a book as the object to be detected. In the function of Lidar following, the object to be detected needs to be higher than the scanning height of the Lidar. Only in this way the Lidar mounted on the MentorPi can effectively scan its position information. Then, the MentorPi moves forward. When the obstacle is detected, the MentorPi will automatically turn to avoid it.

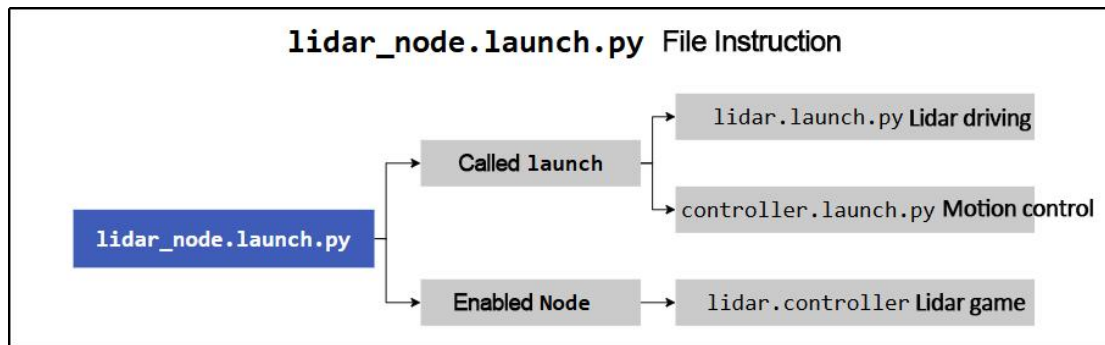


### 3. Program Analysis

#### 3.1 Launch File

The launch file is located in:

/home/ubuntu/ros2\_ws/src/app/launch/lidar\_node.launch.py



```

1 import os
2
3 from launch_ros.actions import Node
4 from launch.conditions import IfCondition
5 from launch import LaunchDescription, LaunchService
6 from launch.substitutions import LaunchConfiguration
7 from launch.launch_description_sources import PythonLaunchDescriptionSource
8 from launch.actions import IncludeLaunchDescription, OpaqueFunction, GroupAction, DeclareLaunchArgument
9
10 def launch_setup(context):
11     compiled = os.environ['need_compile']
12     debug = LaunchConfiguration('debug', default='false')
13     debug_arg = DeclareLaunchArgument('debug', default_value=debug)
14     if compiled == 'True':
15         controller_package_path = get_package_share_directory('controller')
16         peripherals_package_path = get_package_share_directory('peripherals')
17     else:
18         controller_package_path = '/home/ubuntu/ros2_ws/src/driver/controller'
19         peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'
20
21     lidar_controller_node = GroupAction([
22         IncludeLaunchDescription(
23             PythonLaunchDescriptionSource(
24                 os.path.join(peripherals_package_path, 'launch/lidar.launch.py')),
25                 condition=IfCondition(debug),
26             ),
27
28         IncludeLaunchDescription(
29             PythonLaunchDescriptionSource(
30                 os.path.join(controller_package_path, 'launch/controller.launch.py')),
31                 condition=IfCondition(debug),
32             ),
33
34         Node(
35             package='app',
36             executable='lidar_controller',
37             output='screen',
38         ),
39     ])
  
```

#### ◆ Set the storage path

Retrieve the paths for the two packages: peripherals and controller.

```

14 if compiled == 'True':
15     controller_package_path = get_package_share_directory('controller')
16     peripherals_package_path = get_package_share_directory('peripherals')
17 else:
18     controller_package_path = '/home/ubuntu/ros2_ws/src/driver/controller'
19     peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'
  
```

#### ◆ Initiate other Launch files

```

21  lidar_controller_node = GroupAction([
22      IncludeLaunchDescription(
23          PythonLaunchDescriptionSource(
24              os.path.join(peripherals_package_path, 'launch/lidar.launch.py')),
25              condition=IfCondition(debug),
26          ),
27      IncludeLaunchDescription(
28          PythonLaunchDescriptionSource(
29              os.path.join(controller_package_path, 'launch/controller.launch.py')),
30              condition=IfCondition(debug),
31          ),
32      )

```

lidar.launch.py Lidar launch

controller.launch.py Motion control launch

### ◆ Initiate Node

```

34      Node(
35          package='app',
36          executable='lidar_controller',
37          output='screen',
38      ),
39  ])

```

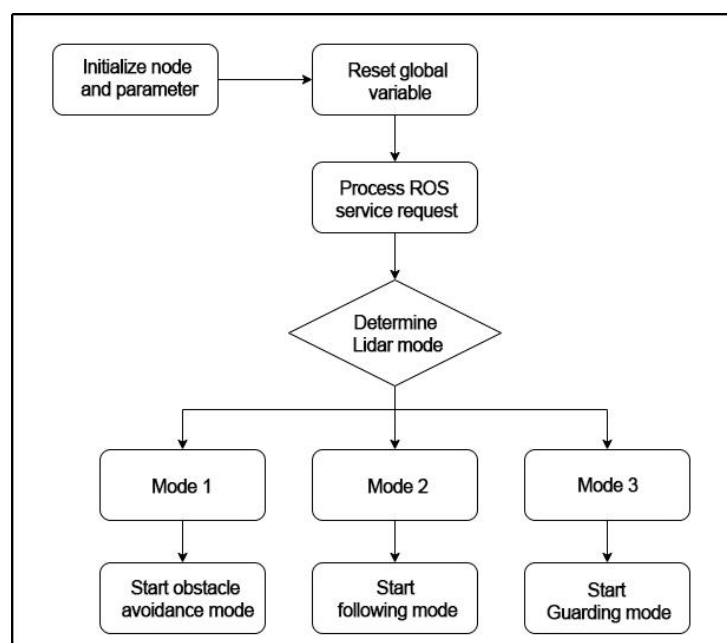
Enable Lidar game node.

## 3.2 Source Code File

The source code of the program is saved in:

**/home/ubuntu/ros2\_ws/src/app/app/lidar\_controller.py**

The process logic of the program based on the implemented effect is shown in the diagram:



## ◆ Initialization

```

25 class LidarController(Node):
26     def __init__(self, name):
27         rclpy.init()
28         super().__init__(name)
29
30         self.name = name
31         self.running_mode = 0
32         self.threshold = 0.6 # meters
33         self.scan_angle = math.radians(360) # radians
34         self.speed = 0.2
35         self.last_act = 0
36         self.timestamp = 0
37         self.angle_data = []

```

self.name: stores the incoming name as the property of the object.

Other attributes include parameters for controlling the Lidar, such as operating mode, threshold, scanning angle, and speed.

self.lock: creates a thread lock for safe access in a multi-threaded environment.

## ◆ Create ROS service

```

46 self.create_service(Trigger, '/enter', self.enter_srv_callback) # 进入游戏(enter the game)
47 self.create_service(Trigger, '/exit', self.exit_srv_callback) # 退出游戏(exit the game)
48 self.create_service(SetInt64, '/set_running', self.set_running_srv_callback) # 开启游戏(start the game)
49 Heart(self, self.name + '/heartbeat', 5, lambda : self.exit_srv_callback(request=Trigger.Request(), response=Trigger.Response())) # 心跳包(heartbeat package)
50 self.create_service(SetFloat64List, '/set_param', self.set_parameters_srv_callback) # 参数设置(set parameter)

```

Enter game: A ROS service named `"/enter"` is created with the type `Trigger`, and the callback function is `enter_srv_callback`. When this service is called, it will execute the `enter_srv_callback` function.

Exit game: A ROS service named `"/exit"` is created with the type `Trigger`, and the callback function is `exit_srv_callback`. When this service is called, it will execute the `exit_srv_callback` function.

Enable game: A ROS service named `"/set_running"` is created with the type `SetInt64`, and the callback function is `set_running_srv_callback`. When this service is called, it will execute the `set_running_srv_callback` function.

Set parameters: A ROS service named `"/set_param"` is created with the type `SetFloat64List`, and the callback function is `set_parameters_srv_callback`.

When this service is called, it will execute the `set_parameters_srv_callback` function.

## ◆ Lidar obstacle avoidance

```

169         if self.running_mode == 1 and self.timestamp <= time.time():
170             left_nonzero = left_range.nonzero()
171             right_nonzero = right_range.nonzero()
172             left_nonan = np.isfinite(left_range[left_nonzero])
173             right_nonan = np.isfinite(right_range[right_nonzero])
174             min_dist_left = left_range[left_nonzero][left_nonan]
175             min_dist_right = right_range[right_nonzero][right_nonan]
176             # 取左右最近的距离(Take the nearest distance left and right)
177             if len(min_dist_left) > 1 and len(min_dist_right) > 1:
178                 min_dist_left = min_dist_left.min()
179                 min_dist_right = min_dist_right.min()
180                 if min_dist_left <= self.threshold and min_dist_right > self.threshold: # 左侧有障碍(there is obstacle at left)
181                     twist.linear.x = self.speed / 6 # 转弯时向前速度尽量小(keep the forward velocity as low as possible when turning)
182                     max_angle = math.radians(90) # 转90度(turn 90 degree)
183                     w = self.speed * 6.0 # 转弯速度尽量大(keep the turning speed as high as possible)
184                     twist.angular.z = -w
185                     if self.last_act != 0 and self.last_act != 1:
186                         twist.angular.z = w
187                     self.last_act = 1
188                     self.mecanum_pub.publish(twist)
189                     self.timestamp = time.time() + (max_angle / w / 2)
190                 elif min_dist_left <= self.threshold and min_dist_right <= self.threshold: # 两侧都有障碍
191                     twist.linear.x = self.speed / 6 # 转弯时向前速度尽量小(keep the forward velocity as low as possible when turning)
192                     w = self.speed * 6.0 # 转弯速度尽量大(keep the turning speed as high as possible)
193                     twist.angular.z = w
194                     self.last_act = 3
195                     self.mecanum_pub.publish(twist)
196                     # 转180度
197                     self.timestamp = time.time() + (math.radians(180) / w / 2)
198                 elif min_dist_left > self.threshold and min_dist_right <= self.threshold: # 右侧有障碍(there is obstacles at right)
199                     twist.linear.x = self.speed / 6 # 转弯时向前速度尽量小(keep the forward velocity as low as possible when turning)
200                     max_angle = math.radians(90) # 转90度 turn 90 degrees
201                     w = self.speed * 6.0 # 转弯速度尽量大(keep the turning speed as high as possible)
202                     twist.angular.z = w
203                     if self.last_act != 0 and self.last_act != 2:
204                         twist.angular.z = -w
205                     self.last_act = 2
206                     self.mecanum_pub.publish(twist)
207                     self.timestamp = time.time() + (max_angle / w / 2)
208             else: # 没有障碍(none obstacle)

```

The code detects the minimum distances on the left and right sides of the robot, stored in variables `min_dist_left` and `min_dist_right`, respectively. Based on the relationship between the minimum distances and a threshold value `self.threshold`, the code determines the presence of obstacles on either side. If there is an obstacle on the left side, the robot will turn right by 90 degrees. If there are obstacles on both sides, the robot will turn around by 180 degrees. If there is an obstacle on the right side, the robot will turn left by 90 degrees. If there is no obstacle, the robot will continue moving forward. To avoid repetitive turns, the code keeps track of the last action performed by the robot in the variable `self.last_act`.